

AI_Project Documentation

رادین_طباطبائی

فاز ۱ و ۲:

این پروژه سامانه‌ای هوشمند برای مسیریابی خودروهای حمل بار در شبکه‌های شهری است. هدف یافتن مسیر بهینه (کم‌هزینه‌ترین) از نقطه شروع به مقصد با در نظر گرفتن هزینه‌های متغیر عبور از خانه‌های مختلف نقشه است.

در فاز اول از الگوریتم **Uniform Cost Search (UCS)** که یک الگوریتم جستجوی بدون اطلاعات اکتشافی است استفاده می‌شود. در فاز دوم با بهره‌گیری از الگوریتم **A*** و تابع هیوریستیک فاصله منتهن، جستجو با هدایت اکتشافی انجام می‌شود.

پیاده‌سازی بر اساس اصول شی‌گرایی طراحی شده و شامل کلاس‌های زیر است:

2.1. کلاس Node

نمایش‌دهنده یک گره در فضای جستجو است:

- **position**: موقعیت گره در نقشه (x, y)
- **g_cost**: هزینه واقعی مسیر از شروع تا این گره
- **h_cost**: هزینه تخمینی از این گره تا هدف (هیوریستیک)
- **parent**: گره والد برای بازسازی مسیر
- **action**: حرکتی که به این گره منجر شده
- **time_mod**: زمان مدولو 30 برای مدیریت خانه‌های Z

متدهای کلیدی:

- $f_cost() = g(n) + h(n)$ محاسبه $f_cost()$
- $get_state(x, y, timemod 30)$: بازگشت حالت منحصر به فرد

2.2. کلاس PriorityQueue

صف اولویت برای مدیریت frontier در الگوریتم‌های جستجو:

- $push(node)$: افزودن گره و مرتب‌سازی بر اساس $f(n)$
- $pop()$: برداشتن گره با کمترین $f(n)$
- $is_empty()$: بررسی خالی بودن صف

2.3. کلاس Map

نمایش نقشه و منطق محیط:

- **grid**: ماتریس نقشه

- **start/goal**: موقعیت شروع و هدف
- **z_cells**: مجموعه خانه‌های با محدودیت زمانی

متدهای کلیدی:

- `get_cell_cost(pos, time)`: محاسبه هزینه ورود به خانه با توجه به زمان
 - خانه‌های عددی: هزینه برابر عدد
 - خانه Z: اگر $15 < T \bmod 30$ هزینه 1، وگرنه 15
 - S و G: هزینه 1
- `get_neighbors(pos)`: برگرداندن همسایه‌های معتبر (UP, DOWN, LEFT, RIGHT و STAY در صورت وجود Z)
- `manhattan_distance(pos1, pos2)`: محاسبه فاصله منتهن

2.4. کلاس SearchEngine

موتور اجرای الگوریتم‌های جستجو:

- `uniform_cost_search()`: پیاده‌سازی UCS
- `a_star_search()`: پیاده‌سازی A*
- `reconstruct_path()`: بازسازی مسیر از گره هدف

3. الگوریتم‌های پیاده‌سازی شده

3.1. Uniform Cost Search (UCS)

ایده: بسط گره‌ها بر اساس کمترین هزینه تجمعی $g(n)$ بدون استفاده از هیوریستیک.

مراحل:

1. شروع از گره S با $g(S) = 0$
2. در هر تکرار، گره با کمترین $g(n)$ از frontier انتخاب می‌شود
3. اگر گره هدف باشد، مسیر بازسازی می‌شود
4. گره بسط داده شده به visited اضافه می‌شود
5. همسایه‌های گره بررسی و به frontier اضافه می‌شوند

مدیریت حالت: برای جلوگیری از حلقه‌های بی‌نهایت در خانه‌های Z، حالت به صورت $(x, y, T \bmod 30)$ ذخیره می‌شود.

پیچیدگی زمانی: $O(b^d)$ که b تعداد شاخه‌ها و d عمق جواب است.

3.2. A* Search

ایده: بسط گره‌ها بر اساس $f(n) = g(n) + h(n)$ که $h(n)$ تخمین هزینه باقیمانده تا هدف است.

تابع هیوریستیک: فاصله منتهن

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

این هیوریستیک *admissible* است زیرا هرگز هزینه واقعی را بیش‌برآورد نمی‌کند (حداقل هزینه ورود به هر خانه 1 است).

مراحل: مشابه UCS با تفاوت که مرتب‌سازی بر اساس $f(n)$ است نه فقط $g(n)$.

مزیت: با هدایت جستجو به سمت هدف، تعداد گره‌های بسط داده شده کاهش می‌یابد.

پیچیدگی زمانی: در بدترین حالت $O(b^d)$ اما در عمل با هیوریستیک خوب بسیار کارآمدتر از UCS.

4. ساختار داده‌ها

4.1. صف اولویت (Priority Queue)

- پیاده‌سازی با لیست و مرتب‌سازی
- در هر $push$ لیست مرتب می‌شود
- pop اولین عنصر (کمترین $f(n)$) را برمی‌گرداند

4.2. مجموعه $visited$

- از Set پایتون استفاده شده
- کلید: $(x, y, \text{timemod } 30)$
- جلوگیری از بازبینی حالت‌های تکراری

4.3. گره‌ها

- هر گره اطلاعات موقعیت، هزینه، والد و عمل را ذخیره می‌کند
- اتصال گره‌ها از طریق $pointer$ به والد امکان بازسازی مسیر را فراهم می‌کند

5. نتایج و مقایسه

تست 1: نقشه ساده بدون Z

3 3

S 1 1

9 8 1

9 9 G

مسیر	تعداد گره های بسط داده شده	هزینه	الگوریتم
------	----------------------------	-------	----------

UCS	4	8	RIGHT, RIGHT, DOWN, DOWN
A*	4	8	RIGHT, RIGHT, DOWN, DOWN

تحلیل: در این نقشه هر دو الگوریتم مسیر بهینه را میابند.
با 2 گره کمتر به جواب می‌رسد زیرا هیوریستیک جستجو را به سمت هدف هدایت می‌کند. A*

تست 2: نقشه با خانه‌های
Z

2 4

S Z 9 9

9 Z Z G

مسیر	تعداد گره های بسط داده شده	هزینه	الگوریتم
RIGHT, DOWN, RIGHT, RIGHT	15	4	UCS
RIGHT, DOWN, RIGHT, RIGHT	10	4	A*

تحلیل: بر اینجا عامل باید تصمیم بگیرد که آیا از Z عبور کند یا دور بزند. با توجه به هزینه بالای Z در زمان‌های خاص، عامل مسیر پایینی را انتخاب می‌کند. تعداد گره‌های بسط داده شده در UCS به دلیل بررسی شاخه‌های نامرتبط بیشتر است.

6. راهنمای اجرا (README)

ورودی را طبق فرمت زیر وارد کنید:

- خط اول: ابعاد N و M
- خطوط بعدی: ماتریس نقشه

2. خروجی شامل هزینه، دنباله حرکت‌ها و تعداد گره‌های بسط داده شده برای هر دو فاز نمایش داده می‌شود.

مثال ورودی:

```
3 3
S 1 1
9 8 1
9 9 G
```

خروجی:

```
=== phase1: Uniform Cost Search ===
Cost: 4 min
Actions: ['RIGHT', 'RIGHT', 'DOWN', 'DOWN']
Expanded nodes: 8
```

```
=== phase2: A* Search ===
Cost: 4 min
Actions: ['RIGHT', 'RIGHT', 'DOWN', 'DOWN']
Expanded nodes: 6
```

فاز ۳:

در فازهای پیشین، مسئله یافتن مسیر بهینه برای یک عامل واحد بررسی شد. در فاز سوم با مسئله‌ای پیچیده‌تر روبه‌رو هستیم: چندین خودروی توزیع $(S_1, S_2, \dots)$ و چندین مقصد $(G_1, G_2, \dots)$ وجود دارد و باید تصمیم بگیریم هر مقصد به کدام خودرو تخصیص یابد تا کار توزیع در کمترین زمان ممکن کامل شود.

این مسئله از خانواده مسائل تخصیص و زمان‌بندی است که در حالت کلی **NP-Hard** محسوب می‌شود. به همین دلیل به جای جستجوی کامل، از یک الگوریتم فراالگشافی (Metaheuristic) یعنی الگوریتم ژنتیک استفاده می‌کنیم که جوابی نزدیک به بهینه را در زمان معقول ارائه می‌دهد.

1.1. تابع هدف

هدف کمینه‌سازی **Makespan** است؛ یعنی بیشترین زمان کاری در میان همه عامل‌ها باید حداقل شود:

$$\text{Makespan} = \max(T_1, T_2, \dots, T_k)$$

که T_i مجموع زمان مسیرهای طی‌شده توسط عامل i است. این معیار باعث می‌شود بار کاری بین خودروها متوازن شود و هیچ خودرویی بیش از حد معطل نماند.

1.2. ساده‌سازی مجاز

طبق صورت مسئله، در این فاز مجاز هستیم زمان مسیر بین دو نقطه را با مسیر یابی واقعی محاسبه کنیم. در پیاده‌سازی فعلی از موتور جستجوی فاز 2 (A^*) برای محاسبه دقیق هزینه مسیر بین نقاط استفاده شده است.

2. ساختار کلاسی پروژه

پیاده‌سازی مطابق اصول شی‌گرایی و با اتکا به ماژول فازهای 1 و 2 ($Faz1_2$) انجام شده است.

2.1. کلاس Agent

نمایش‌دهنده یک عامل (خودروی توزیع):

- **id**: شناسه عددی عامل
- **start_position**: موقعیت اولیه عامل (x, y)

2.2. کلاس Chromosome

نماینده یک راه‌حل کاندید در فضای جستجوی ژنتیک:

- **genes**: لیستی به طول تعداد اهداف. مقدار خانه مشخص می‌کند مقصد به کدام عامل تخصیص داده شده است.
- **fitness**: برازندگی کروموزوم $\left(\frac{1}{makespan}\right)$
- **makespan**: بیشینه زمان کاری عامل‌ها
- **agent_costs**: دیکشنری هزینه تجمعی هر عامل

متد **copy ()** برای تولید نسخه مستقل از کروموزوم (لازم برای نخبه‌گرایی و *crossover*).

نمایش ژنی (Encoding)

اگر 4 مقصد و 2 عامل داشته باشیم، کروموزومی مثل $[0, 1, 0, 1]$ یعنی:

- مقصد 0 و مقصد 2 به عامل 0
- مقصد 1 و مقصد 3 به عامل 1

این نمایش مستقیم (*Direct Encoding*) است و فضای جستجو شامل n^k حالت می‌شود (k : تعداد عامل‌ها، n : تعداد اهداف).

2.3. کلاس GeneticAlgorithm

موتور اصلی الگوریتم ژنتیک که تمام عملگرها را پیاده‌سازی می‌کند. پارامترهای قابل تنظیم:

توضیح	مقدار پیش فرض	پارامتر
اندازه جمعیت	100	population_size
تعداد نسل‌ها	200	generations

نرخ جهش	0.1	mutation_rate
نرخ ترکیب	0.8	crossover_rate

همچنین یک `path_cache` برای ذخیره هزینه مسیرهای محاسبه شده دارد تا از محاسبه تکراری A^* جلوگیری شود.

3. طراحی الگوریتم ژنتیک

3.1. محاسبه هزینه مسیر (`calculate_path_cost`)

برای هر جفت (مبدأ، مقصد):

1. ابتدا کش بررسی می‌شود.
2. در صورت نبود، یک شیء Map موقت ساخته شده و `goal/start` آن تنظیم می‌شود.
3. الگوریتم A^* اجرا و هزینه واقعی مسیر برگردانده می‌شود.
4. نتیجه در کش ذخیره می‌شود.

این روش تضمین می‌کند هزینه‌ها واقعی و سازگار با محیط (شامل خانه‌های Z) باشند، نه صرفاً تخمین هندسی.

3.2. تابع برازندگی (`evaluate_chromosome`)

برای هر کروموزوم:

1. هزینه و موقعیت هر عامل مقداردهی اولیه می‌شود.
2. اهداف به ترتیب پیمایش می‌شوند؛ هر هدف بر اساس ژن مربوطه به عاملی تخصیص می‌یابد.
3. هزینه مسیر از موقعیت فعلی عامل تا مقصد محاسبه و به هزینه آن عامل افزوده می‌شود.
4. موقعیت عامل به مقصد جدید به‌روزرسانی می‌شود (مدل بازدید زنجیره‌ای).
5. در پایان:

$$makespan = \max_i (agent_costs_i), fitness = \frac{1}{makespan}$$

اگر مسیری غیر قابل دسترس باشد، `makespan` برابر بی‌نهایت و `fitness` صفر می‌شود.

3.3. عملگرهای ژنتیک

انتخاب (Selection) — روش مسابقه‌ای (Tournament):

از میان 5 کروموزوم تصادفی، بهترین (کمترین `makespan`) انتخاب می‌شود. این روش فشار انتخابی متعادلی ایجاد می‌کند و از همگرایی زودرس جلوگیری می‌کند.

ترکیب (Crossover) — تک‌نقطه‌ای (Single-Point):

با احتمال `crossover_rate`، یک نقطه برش تصادفی انتخاب شده و ژن‌های دو والد قبل و بعد از آن نقطه با هم تعویض می‌شوند:

$$child_1 = parent_1[:p] + parent_2[p:]$$

جهش (Mutation):

هر ژن با احتمال `mutation_rate` به یک عامل تصادفی جدید تغییر می‌کند. این عملگر تنوع ژنتیکی را حفظ کرده و از گیر افتادن در بهینه محلی جلوگیری می‌کند.

3.4. حلقه اصلی و نخبه‌گرایی (Elitism)

در هر نسل:

1. جمعیت بر اساس `makespan` مرتب می‌شود.
2. 10% برتر (نخبگان) مستقیماً به نسل بعد منتقل می‌شوند تا بهترین جواب از بین نرود.
3. بقیه جمعیت از طریق انتخاب، ترکیب و جهش تولید می‌شوند.
4. فرزندان ارزیابی شده و به جمعیت جدید افزوده می‌شوند.

این چرخه تا اتمام `generations` ادامه می‌یابد و بهترین کروموزوم نهایی به عنوان جواب بازگردانده می‌شود.

4. ساختار داده‌ها

- **جمعیت (Population):** لیستی از اشیاء `Chromosome`.
- **کش مسیر (path_cache):** دیکشنری با کلید `(start, goal)` و مقدار هزینه مسیر. این بهینه‌سازی، تعداد فراخوانی‌های پر هزینه `A*` را به شدت کاهش می‌دهد.
- **agent_costs:** دیکشنری نگهداری هزینه تجمعی هر عامل برای محاسبه `makespan`.
- **تاریخچه makespan:** لیستی برای ثبت بهترین `makespan` در هر نسل (برای تحلیل همگرایی).

5. پردازش ورودی و خروجی

5.1. تجزیه ورودی (parse_phase3_input)

نقشه پیمایش شده و خانه‌های `S` {شماره} به عنوان عامل و `G` {شماره} به عنوان مقصد شناسایی می‌شوند. عامل‌ها و اهداف بر اساس شماره مرتب می‌شوند تا ترتیب قطعی حفظ شود.

5.2. نمایش جواب (print_solution)

خروجی شامل بهترین `makespan`، تخصیص اهداف به هر عامل (با برچسب و مختصات) و زمان مسیر هر عامل است.

6. نتایج و تحلیل

تست نمونه

```
4 4
S1 1 1 G1
1 1 1 1
S2 1 G2 1
```


خروجی:

```
Best makespan (minutes): 3.00
S1 at (0, 0) assigned targets: ['G1'] -> coords: [(0, 3)]
S2 at (2, 0) assigned targets: ['G2'] -> coords: [(2, 2)]
S1 route time = 3.0 minutes
S2 route time = 2.0 minutes
```

تحلیل: الگوریتم به‌درستی $G1$ را به $S1$ و $G2$ را به $S2$ تخصیص داده است. این تخصیص بهینه است زیرا:

- اگر هر دو هدف به یک عامل داده می‌شد، $makespan$ افزایش می‌یافت.
- تخصیص فعلی بار را متوازن می‌کند: $T_1 = 3$ ، $T_2 = 2$ و در نتیجه $Makespan = \max(3, 2) = 3$.

تحلیل همگرایی

نمودار تاریخچه $makespan$ معمولاً نشان می‌دهد:

- در نسل‌های ابتدایی، $makespan$ به‌سرعت کاهش می‌یابد.
- پس از چند ده نسل، جواب به مقدار بهینه همگرا شده و ثابت می‌ماند.
- نخبه‌گرایی تضمین می‌کند منحنی هرگز صعودی نشود (یکنوازی نزولی).

فاز ۴:

در فازهای پیشین، الگوریتم‌های A^* و UCS برای یافتن مسیر بهینه در فضای حالت استفاده شد. این الگوریتم‌ها از ساختارهای داده‌ای مانند صف اولویت و مجموعه بازدیدشده استفاده می‌کنند که در مسائل با فضای حالت بزرگ، حافظه قابل توجهی مصرف می‌کنند.

در فاز 4 با دو تغییر اساسی رویه‌رو هستیم:

1. **محدودیت حافظه:** به‌جای A^* که تمام گره‌های بازدیدشده را ذخیره می‌کند، باید از الگوریتم IDA^* (*Iterative Deepening A*) استفاده کنیم که با جستجوی عمقی تکراری و آستانه‌های افزایشی، حافظه را به $O(bd)$ کاهش می‌دهد (b شاخه‌زایی، d عمقی).
2. **معرفی پل‌ها (Bridges):** خانه‌های Z حذف شده و به‌جای آن خانه‌های پل B_K معرفی می‌شوند. هر پل دارای شماره K است که هزینه ورود اولیه به آن را مشخص می‌کند. حرکت روی خانه‌های متصل همان پل (با شماره یکسان) هزینه صفر دارد، اما خروج و ورود مجدد نیازمند پرداخت دوباره هزینه K است.

این تغییرات، ساختار حالت ($State$) و منطق محاسبه هزینه را تغییر می‌دهند و نیاز به طراحی دقیق‌تری دارند.

2. تحلیل مسئله**2.1 فضای حالت جدید**

در فازهای قبلی، حالت به‌صورت $(x, y, T \bmod 30)$ بود (برای مدیریت چرخه‌های Z). در فاز 4:

- خانه‌های Z حذف شده‌اند، پس دیگر نیازی به ردیابی زمان در حالت نیست.
- اما باید بدانیم عامل روی کدام پل قرار دارد (یا هیچ‌کدام).

حالت جدید:

$$State = (x, y, current_bridge)$$

که $current_bridge$ می‌تواند:

- عدد صحیح k باشد (عامل روی پل B_k است)
- $None$ باشد (عامل روی پل نیست)

2.2. قوانین محاسبه هزینه

تابع $get_cell_cost_phase4$ هزینه ورود به هر خانه را بر اساس موقعیت فعلی و پل جاری محاسبه می‌کند:

هزینه ورود	شرایط	نوع خانه مقصد
0	عامل روی همان پل B_k است	B_k
k	عامل روی پل دیگری یا هیچ‌کدام است	B_k
1	همیشه	$G \cup S$
مقدار عدد	همیشه	عدد $(9-1)$
∞	-	خارج از نقشه

این منطق تضمین می‌کند:

- حرکت پیوسته روی یک زنجیره پل بدون هزینه اضافی است.
- خروج از پل و بازگشت مجدد نیازمند پرداخت دوباره است.

3. الگوریتم ID^* (Iterative Deepening A)

3.1. ایده اصلی

ID^* ترکیبی از:

- جستجوی عمقی (DFS): استفاده از پشته (Stack) یا بازگشت (Recursion) به‌جای صف اولویت.
- محدودیت آستانه (Cutoff): در هر تکرار، فقط گره‌هایی بررسی می‌شوند که $f(n) = g(n) + h(n) \leq cutoff$.

الگوریتم به‌صورت تکراری اجرا می‌شود:

1. آستانه اولیه برابر $h(start)$ است.

2. جستجوی عمقی محدود به آستانه انجام می‌شود.
3. اگر هدف یافت نشد، آستانه به کمترین مقدار f که از $cutoff$ تجاوز کرد، افزایش می‌یابد.
4. این چرخه تا یافتن هدف یا عدم وجود مسیر ادامه می‌یابد.

مزایا:

- حافظه خطی $O(bd)$ به‌جای نمایی.
- بهینه بودن (مانند A^*) با هیوریستیک قابل قبول.

معایب:

- بازبینی مکرر گره‌ها در تکرارهای مختلف.
- زمان اجرا می‌تواند بیشتر از A^* باشد.

3.2. هیوریستیک

همانند فازهای قبل، از فاصله منتهن (Manhattan Distance) استفاده می‌شود:

$$h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$$

این هیوریستیک:

- قابل قبول (Admissible) است: هرگز هزینه واقعی را بیش‌برآورد نمی‌کند.
- سازگار (Consistent) است: برای هر پال (n, n') داریم $h(n) \leq c(n, n') + h(n')$.

4. ساختار پیاده‌سازی

4.1. کلاس BridgeNode

نمایش‌دهنده یک گره در درخت جستجو:

```
class BridgeNode:
    def __init__(self, position, g_cost=0.0, h_cost=0.0,
                 parent=None, action=None, current_bridge=None):
        self.position = position          # (x, y)
        self.g_cost = g_cost              # هزینه تجمعی از ریشه
        self.h_cost = h_cost              # تخمین هیوریستیک تا هدف
        self.parent = parent              # گره والد (برای بازسازی مسیر)
        self.action = action              # اقدام منجر به این گره
        self.current_bridge = current_bridge # شماره پل فعلی یا None
```

متد کلیدی:

- $get_state()$: برمی‌گرداند $(x, y, current_bridge)$ برای بررسی تکراری.
- $f_cost()$: محاسبه $f(n) = g(n) + h(n)$ برای مقایسه با آستانه.

4.2. تابع `a_star_phase4`

پیدامسازی A^* استاندارد برای این فاز (برای مقایسه با IDA^*). ساختار مشابه فازهای قبل، اما با:

- استفاده از `BridgeNode` به جای `Node`.
- فراخوانی `get_cell_cost_phase4`.

برای محاسبه هزینه لبه‌ها.

4.3. کلاس `IDASolver`

قلب فاز 4 که شامل دو متد اصلی است:

1. `search()`: حلقه بیرونی که آستانه (`cutoff`) را مدیریت می‌کند.
2. `dfs()`: تابع بازگشتی که جستجوی عمقی محدود را انجام می‌دهد.

نکته مهم در `dfs`:

برای جلوگیری از لوپ‌های بی‌نهایت در مسیرهای فعلی، یک مجموعه `visited` محلی برای هر مسیر عمقی استفاده می‌شود.

5. تحلیل عملکرد و مقایسه

5.1. مقایسه A^* و IDA^* در فاز 4

ویژگی	A^*	IDA^*
مصرف حافظه	بالا (نگهداری تمام گره‌های باز شده در <code>Open/Closed list</code>)	بسیار پایین (فقط گره‌های مسیر فعلی)
زمان اجرا	بهینه (کمترین تعداد گره‌های بسط داده شده)	بیشتر (به دلیل تکرار جستجو در عمق‌های کمتر)
بهینه بودن جواب	دارد (اگر قابل قبول باشد)	دارد (اگر قابل قبول باشد)
پیدامسازی	پیچیدمتر (نیاز به <code>PriorityQueue</code>)	ساده‌تر (بازگشتی)

5.2. تعداد گره‌های بسط داده شده (`Expanded Nodes`)

در تست‌های انجام شده:

- در نقشه‌های کوچک، IDA^* گره‌های بیشتری بسط می‌دهد زیرا چندین بار همان گره‌ها را با آستانه‌های مختلف می‌بیند.
- در نقشه‌های بزرگ و حافظه محدود، IDA^* تنها راه حل ممکن است، در حالی که A^* ممکن است با خطای کمبود حافظه مواجه شود.

5.3. تحلیل حالت‌ها (`States`)

تعداد کل حالت‌های ممکن در این فاز برابر است با:

$$\text{Number of Cells} \times (\text{Number of Bridges} + 1)$$

اضافه شدن پل‌ها فضای حالت را نسبت به حالت عادی بزرگتر می‌کند، اما چون چرخه زمان Z حذف شده، کل فضای حالت نسبت به فاز 1 و 2 کوچکتر و مدیریت آن با IDA^* کارآمدتر است.

1. ورودی را مطابق فرمت نقشه (شامل B_k) وارد کنید.
2. برنامه ابتدا با $IDASolver$ مسیر را پیدا کرده و هزینه، لیست اقدامات و تعداد گره‌های بسط داده شده را نمایش می‌دهد.

مثال تست:

```
3 4
S  9 9 9
B4 B4 B4 1
9  9 9 G
```

خروجی مورد انتظار:

```
Cost: 6 min
Actions: ['DOWN', 'RIGHT', 'RIGHT', 'RIGHT', 'DOWN']
Expanded nodes: 15
```

(توضیح: هزینه 6 شامل 1 (ورود به 1) + 0 + 0 + 4 (ورود به G) نیست، بلکه ورود به G همواره 1 است: $1 + 0 + 0 + 1 = 4$ ؛ خیر؛ با توجه به صورت مسئله، حرکت S به B4 هزینه 4 دارد، سپس روی B4 ها هزینه 0، سپس ورود به 1، هزینه 1 و ورود به G هزینه 1. جمعاً: $4 + 0 + 0 + 1 + 1 = 6$).

توضیحات بیشتر پیاده سازی :

فاز ۱-۲:

2.1. کلاس Node

نمایش دهنده یک حالت در فضای جستجو:

```
class Node:
    position: (x, y)          # موقعیت فعلی
```

```

g_cost: float          # هزینه تجمعی از ریشه
h_cost: float          # تخمین هیوریستیک (فقط در
parent: Node           # گره والد برای بازسازی مسیر
action: str            # اقدام منجر به این گره
time_mod: int          #  $T \bmod 30$  برای مدیریت خانه های Z

```

توابع کلیدی:

- $f_cost()$: محاسبه $f(n) = g(n) + h(n)$ برای مقایسه در صف
- $get_state()$: برمی گرداند $(x, y, T \bmod 30)$ برای جلوگیری از بازدید تکراری

چرا **time_mod**؟ در حضور Z، اگر فقط (x, y) ذخیره شود، الگوریتم ممکن است به حلقه بیفتد چون هزینه ورود به Z به زمان بستگی دارد. با نگهداری $T \bmod 30$ ، حالت های متمایز شناسایی می شوند.

2.2. کلاس PriorityQueue

صف اولویت ساده برای مدیریت گره های باز شده:

```

push(node): f_cost اضافه کردن گره و مرتب سازی بر اساس
pop(): f_cost برداشتن گره با کمترین

```

توجه: در پیاده سازی واقعی، از heapq برای $O(\log n)$ استفاده می شود، اما اینجا برای سادگی از sort استفاده شد.

مدیریت نقشه و قوانین محیطی:

توابع کلیدی:

```

get_cell_cost(pos, time)

```

محاسبه هزینه ورود به هر خانه.

```

if cell == 'Z':
    time_mod = int(time) % 30
    return 1.0 if time_mod < 15 else 15.0

```

این تابع قلب منطق محیطی است و باعث می شود عامل در برخی مواقع ترجیح دهد STAY کند تا از پرداخت ۱۵ دقیقه جریمه اجتناب نماید.

```

get_neighbors(pos)

```

لیست حرکات ممکن را برمی گرداند:

```

directions = {'RIGHT': (0,1), 'LEFT': (0,-1), 'UP': (-1,0), 'DOWN': (1,0)}
if self.z_cells: # وجود دارد Z اگر
    directions['STAY'] = (0,0)

```

نکته: STAY فقط وقتی فعال می‌شود که Z در نقشه وجود داشته باشد، زیرا در غیر این صورت هیچ دلیلی برای توقف نیست.

`manhattan_distance(pos1, pos2)`

هیورستیک استفاده شده در $*A$:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

چرا قابل قبول (Admissible) است؟ زیرا کمترین هزینه واقعی برای رسیدن از n به هدف حداقل به اندازه فاصله منتهی است (چون کمترین هزینه هر خانه ۱ است).

2.4. کلاس SearchEngine

موتور اصلی جستجو:

الگوریتم UCS

```
def uniform_cost_search():
    frontier = PriorityQueue()
    frontier.push(start_node)
    visited = set()

    while not frontier.is_empty():
        current = frontier.pop()

        if current.position == goal:
            return reconstruct_path(current)

        if current.get_state() in visited:
            continue

        visited.add(current.get_state())
        expanded_nodes += 1

        for action, new_pos in get_neighbors():
            move_cost = get_cell_cost(new_pos, current.g_cost)
            new_node = Node(new_pos, g_cost=current.g_cost + move_cost, ...)
            frontier.push(new_node)
```

ویژگی‌ها:

- $h(n) = 0$ برای تمام گره‌ها (بدون دانش قبلی)
- بهینه است زیرا همواره گره با کمترین g را بسط می‌دهد
- تعداد گره‌های بسط داده شده بیشتر از $*A$ است

الگوریتم $*A$

```
def a_star_search(heuristic_func):
    start_node.h_cost = heuristic_func(start)
```

```
# است، با این تفاوت UCS بقیه کد مشابه
new_node.h_cost = heuristic_func(new_pos)
```

تفاوت با UCS:

- اضافه شدن h_cost به هر گره
- مقایسه بر اساس $f(n) = g(n) + h(n)$
- گره‌های کمتری بسط داده می‌شود زیرا هیوریستیک جستجو را هدایت می‌کند

فاز ۳:

نمایش کروموزوم

هر کروموزوم یک راحل کاندید است:

```
genes = [agent_id_for_G1, agent_id_for_G2, ..., agent_id_for_Gn]
```

مثال: اگر ۲ عامل و ۳ هدف داشته باشیم:

```
genes = [0, 1, 0] # G1→Agent0, G2→Agent1, G3→Agent0
```

این نمایش ساده و کارآمد است زیرا:

- طول کروموزوم ثابت است (n)
- عملگرهای ژنتیک (crossover/mutation) به راحتی قابل پیاده‌سازی هستند
- هر کروموزوم یک راحل معتبر است (نیازی به بررسی قیود نیست)

ساختار کلاس‌ها

3.1. کلاس Agent

```
class Agent:
    id: int # شناسه عامل
    start_position: (x, y) # موقعیت شروع
```

3.2. کلاس Chromosome

```
class Chromosome:
    genes: List[int] # لیست تخصیص‌ها
    fitness: float # 1 / makespan
    makespan: float # بیشترین زمان بین عامل‌ها
    agent_costs: Dict[int, float] # زمان هر عامل
```

چرا $fitness = 1 / makespan$ ؟

الگوریتم ژنتیک معمولاً به دنبال بیشینه‌سازی fitness است. با این تعریف، کمینه کردن makespan معادل بیشینه کردن fitness می‌شود.

GeneticAlgorithm کلاس

پارامترهای اصلی

```
population_size = 100      # تعداد کروموزوم‌ها در هر نسل
generations = 200          # تعداد نسل‌ها
mutation_rate = 0.1         # احتمال جهش هر ژن
crossover_rate = 0.8        # احتمال تقاطع
```

محاسبه هزینه مسیر: calculate_path_cost

```
def calculate_path_cost(start, goal):
    cache_key = (start, goal)
    if cache_key in path_cache:
        return path_cache[cache_key]

    temp_map = Map(grid)
    temp_map.start = start
    temp_map.goal = goal

    search_engine = SearchEngine(temp_map)
    result = search_engine.a_star_search()

    if result:
        _, cost, _ = result
        path_cache[cache_key] = cost
        return cost
    else:
        return None
```

نکات کلیدی:

1. **Cache**: از آنجا که ممکن است مسیر بین دو نقطه چندین بار محاسبه شود، نتایج را ذخیره می‌کنیم.
2. استفاده از ***A**: برای دقت بیشتر، از موتور جستجوی فاز ۲ استفاده می‌کنیم (نه فاصله منتهن خام).
3. مدیریت مسیر نامعتبر: اگر ***A** مسیری پیدا نکند، **None** برمی‌گرداند.

ارزیابی کروموزوم: evaluate_chromosome

```
def evaluate_chromosome(chromosome):
    agent_costs = {agent.id: 0.0 for agent in agents}
    agent_positions = {agent.id: agent.start_position for agent in agents}

    for goal_idx, agent_id in enumerate(chromosome.genes):
        goal_pos = goals[goal_idx]
        current_pos = agent_positions[agent_id]

        cost = calculate_path_cost(current_pos, goal_pos)
```

```

if cost is None:
    chromosome.makespan = inf
    chromosome.fitness = 0.0
    return

agent_costs[agent_id] += cost
agent_positions[agent_id] = goal_pos

chromosome.makespan = max(agent_costs.values())
chromosome.fitness = 1.0 / chromosome.makespan

```

منطق:

1. هر عامل از موقعیت شروعش آغاز می‌کند.
2. برای هر هدف در *genes*:
 - مسیر از موقعیت فعلی آن عامل تا هدف محاسبه می‌شود
 - هزینه به زمان تجمعی عامل اضافه می‌شود
 - موقعیت عامل به هدف جدید به‌روز می‌شود
3. *makespan* = بیشترین زمان بین همه عامل‌ها

مثال:

genes = [0, 1, 0]

Agent 0: S1 → G1 (cost=3) → G3 (cost=2) → total=5

Agent 1: S2 → G2 (cost=4) → total=4

Makespan = max(5, 4) = 5

مقداردهی اولیه: *initialize_population*

```

def initialize_population():
    population = []
    for _ in range(population_size):
        chromosome = create_random_chromosome()
        evaluate_chromosome(chromosome)
        population.append(chromosome)
    return population

```

هر کروموزوم تصادفی ایجاد و ارزیابی می‌شود.

انتخاب: *selection*

```

python
def selection(population):
    tournam

```

et = random.sample(population, 5)

```
return min(tournament, key=lambda c: c.makespan)
```

Tournament Selection: انتخاب تصادفی ۵ کروموزوم و برگرداندن بهترین آن‌ها. این کار از انتخاب زودهنگام بهترین‌ها (و گیر کردن در بهینه محلی) جلوگیری می‌کند و تنوع را حفظ می‌کند.

4.5. تقاطع و جهش: *mutate و crossover*

Single-point Crossover:

```
child1_genes = parent1_genes[:cp] + parent2_genes[cp:]
```

```
child2_genes = parent2_genes[:cp] + parent1_genes[cp:]
```

ترکیب ویژگی‌های دو والدین برای ایجاد فرزندان.

Mutation:

```
for i in range(num_goals):
```

```
if random.random() < mutation_rate:
```

```
chromosome_genes[i] = random.randint(0, num_agents - 1)
```

تغییر تصادفی یک تخصیص (مثلاً تغییر هدف $G1$ از عامل ۱ به عامل ۲) برای کاوش فضاهای جدید.

اجرای الگوریتم: *run*

چرخه حیات نسل‌ها:

1. مرتب‌سازی: بهترین رامحل در صدر قرار می‌گیرد.
2. نخبه‌گرایی (**Elitism**): ۱۰٪ برتر جمعیت مستقیماً به نسل بعد می‌روند تا بهترین جواب‌ها از دست نروند.
3. تولید مثل: با استفاده از انتخاب، تقاطع و جهش، بقیه جمعیت (۹۰٪) ایجاد می‌شوند.
4. به‌روزرسانی: نسل جدید جایگزین نسل قدیم می‌شود.

فاز ۴:

۱. تغییرات ساختار مسئله

در این فاز:

- خانه‌های Z حذف شدند → دیگر نیازی به `time_mod` نداریم.
- پل‌های Bk اضافه شدند → حالت سیستم باید شامل «پلی که روی آن هستیم» باشد.
- محدودیت حافظه → باید از الگوریتمی با مصرف حافظه $O(bd)$ استفاده کنیم.

حالت جدید (State):

```
state = (x, y, current_bridge)
```

- `current_bridge = k` اگر روی پل B_k باشیم
- `current_bridge = None` اگر روی پلی نباشی

۲. منطق محاسبه هزینه پل‌ها

تابع `get_bridge_num`

```
def get_bridge_num(cell: str) -> Optional[int]:
    if cell.startswith('B'):
        try:
            return int(cell[1:])
        except ValueError:
            return None
    return None
```

استخراج شماره پل از رشته (مثلاً "B4" → 4).

تابع `get_cell_cost_phase4`

```
def get_cell_cost_phase4(map_obj, pos, current_bridge):
    cell = map_obj.grid[x][y]
    bridge_num = get_bridge_num(cell)

    if bridge_num is not None:
        if current_bridge == bridge_num:
            return 0.0 # حرکت در همان زنجیره پل
        return float(bridge_num) # ورود جدید به پل

    # سایر خانه‌ها
    if cell in ['S', 'G'] or cell.startswith('S') or cell.startswith('G'):
        return 1.0
    if cell.isdigit():
        return float(cell)
    return float('inf')
```

منطق کلیدی:

1. اگر خانه هدف پل نباشد → از پل خارج می‌شویم.
2. اگر خانه هدف همان پلی باشد که روی آن هستیم → هزینه = 0

3. اگر خانه هدف پل دیگری باشد $\rightarrow$ هزینه k جدید

مثال:

مسیر: $S \rightarrow B4 \rightarrow B4 \rightarrow B4 \rightarrow G$

هزینه‌ها: $6 = 1 + 0 + 0 + 4 + 1$

۳. کلاس BridgeNode

```
class BridgeNode:
    position: (x, y)
    g_cost: float
    h_cost: float
    parent: BridgeNode
    action: str
    current_bridge: Optional[int]

    def get_state(self):
        return (self.position[0], self.position[1], self.current_bridge)
```

تفاوت با فاز ۲:

- به جای $current_bridge \rightarrow time_mod$ داریم.
- حالت‌های (x, y, k) و $(x, y, None)$ متفاوت محسوب می‌شوند.

۴. پیاده‌سازی A* با پل‌ها

```
def a_star_phase4(map_obj):
    # موقعیت شروع ممکن است خود یک پل باشد
    start_bridge = get_bridge_num(map_obj.grid[sr][sc])

    start_node = BridgeNode(
        map_obj.start, g_cost=0.0,
        h_cost=heuristic(map_obj.start),
        current_bridge=start_bridge
    )

    while not frontier.is_empty():
        current = frontier.pop()

        # کامل بررسی بازدید با
        state = current.get_state()
        if state in visited:
            continue
        visited.add(state)

        # بسط همسایه‌ها
        for action, npos in map_obj.get_neighbors(current.position):
```

```

        move_cost = get_cell_cost_phase4(map_obj, npos,
current.current_bridge)

        new_g = current.g_cost + move_cost
        new_bridge = get_bridge_num(map_obj.grid[npos[0]][npos[1]])

        new_node = BridgeNode(
            position=npos, g_cost=new_g,
            h_cost=heuristic(npos),
            parent=current, action=action,
            current_bridge=new_bridge
        )

```

نکات:

- استفاده از `get_state()` برای جلوگیری از بازدید مجدد حالت‌های یکسان.
- به‌روزرسانی `current_bridge` برای هر گره جدید.

۵. الگوریتم IDA*

مفهوم

$IDA^* = \text{Iterative Deepening } A^*$

- جستجوی عمقی (DFS) با محدودیت cost-f .
- هر بار آستانه (cutoff) افزایش می‌یابد تا مسیر پیدا شود.
- مصرف حافظه: $O(b^d)$ به جای $O(b^d)$ در A^* .

کلاس IDAStarSolver

تابع اصلی: `search`

```

def search(self):
    start = self.map.start
    start_bridge = get_bridge_num(self.map.grid[start[0]][start[1]])

    cutoff = self.heuristic(start) # آستانه اولیه = h(start)

    while True:
        visited = {(start, start_bridge)}
        path = [(start, None, 0.0, start_bridge)]

        result = self._dfs(path, visited, cutoff)

        if isinstance(result, list):
            # مسیر پیدا شد
            actions = [step[1] for step in result if step[1] is not None]
            cost = result[-1][2]

```

return

urn actions, cost, self.expanded

if result == float('inf'):

#return None هیچ مسیری وجود ندارد